UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

ESCUELA DE INGENIERÍA DE SISTEMAS

Guía Práctica de Laboratorio

“Trabajo de Unidad”

Que se presenta para el curso:

“Investigacion de Operaciones”

Docente:

MSc. Silvia Centella Vildoso

Estudiantes:

Matteo Alejandro Cutipa Peralta

Emerson Adilson Pariguana Yujra

Andre Fernando Valdez Fernandez

Geovani Jesús Lajo Zuela

Milagros Condori Acero

Mario Eduardo Gomez Mamani

Semestre académico:

2026-II

TACNA – PERÚ

1. Resumen ejecutivo

Se propone una aplicación web para consultar manuales técnicos en PDF mediante preguntas en lenguaje natural. La solución utiliza generación aumentada por recuperación (Retrieval-Augmented Generation, RAG) para recuperar fragmentos relacionados con la pregunta antes de generar la respuesta, y Model Context Protocol (MCP) para obtener, cuando corresponda, información adicional mediante herramientas autorizadas de solo lectura. La recuperación documental y la invocación de herramientas son responsabilidades diferentes: la primera aporta evidencia de los manuales y la segunda permite obtener un dato operativo puntual (Anthropic, 2024; Lewis et al., 2020; Model Context Protocol, 2025).

El sistema se organizará sobre una arquitectura hexagonal en Java con Spring Boot. La máquina virtual MATTEO-01 de Microsoft Azure alojará la interfaz web, la API, la cuenta administrativa en memoria, los documentos originales, los metadatos y el servidor MCP. El procesamiento de IA se realizará en MATTEOXPC mediante ProjectSevera, Python, FastAPI y PyTorch. Ambos equipos se comunicarán por una red privada Tailscale. El diseño evita incorporar una base relacional o un clúster de servidores cuando un repositorio de archivos y un índice vectorial local son suficientes para el prototipo.

Se establecen cuatro casos de uso, ocho requerimientos funcionales y once requerimientos no funcionales con criterios de aceptación medibles. La propuesta incluye diagramas de casos de uso, arquitectura lógica, despliegue físico, flujo de información, red y seguridad, así como un plan de implementación, controles de acceso, matriz de riesgos, plan de pruebas y análisis de factibilidad. La disponibilidad de ProjectSevera depende del equipo local; las métricas de latencia, calidad y recuperación serán verificadas durante la Unidad III y no se presentan como resultados obtenidos.

2. Planteamiento del problema

2.1. Situación actual

La consulta de documentación técnica constituye una actividad necesaria para la instalación, configuración, administración y resolución de problemas en sistemas informáticos. Sin embargo, la información requerida suele encontrarse distribuida entre múltiples documentos, versiones y secciones, lo que dificulta localizar instrucciones específicas mediante consultas expresadas en lenguaje natural. Esta problemática adquiere mayor relevancia cuando una respuesta depende de información procedente de varios documentos o requiere mantener el contexto de una conversación.

La magnitud de la recuperación de información documental ha sido estudiada mediante conjuntos de datos especializados. Feng et al. (2020) desarrollaron Doc2Dial, un conjunto de datos que comprende más de 4500 conversaciones fundamentadas en más de 450 documentos pertenecientes a cuatro dominios. Posteriormente, Feng et al. (2021) ampliaron el estudio a conversaciones que requieren información de múltiples documentos mediante MultiDoc2Dial. Estos antecedentes evidencian la complejidad de localizar, relacionar e interpretar información documental durante una interacción conversacional.

Las limitaciones no se restringen a la recuperación de información. Friel et al. (2024) presentaron RAGBench, un conjunto de evaluación de aproximadamente 100 000 ejemplos distribuidos en cinco dominios especializados, incluidos escenarios relacionados con manuales y asistencia técnica. Los autores identificaron dificultades para evaluar de manera consistente la calidad de las respuestas generadas por sistemas RAG, especialmente respecto de la utilización de evidencia y la fundamentación de las afirmaciones.

Por otra parte, la selección de modelos de representaciones vectoriales constituye una dificultad adicional. Muennighoff et al. (2023) evaluaron 33 modelos mediante MTEB, considerando 58 conjuntos de datos distribuidos en ocho categorías de tareas. Sus resultados demostraron que ningún método de embeddings presenta un rendimiento superior en todas las tareas, por lo que la elección del modelo requiere considerar las características del dominio y las consultas que deberá procesar.

En el contexto del presente proyecto, estas dificultades se trasladan a la consulta de manuales técnicos en PDF dentro de un entorno académico controlado. Actualmente no se dispone de una herramienta integrada que, dentro de la infraestructura propuesta, permita administrar los documentos, recuperar información semánticamente relacionada, generar respuestas fundamentadas y consultar datos operativos autorizados mediante MCP.

El proyecto parte de una colección experimental prevista de 20 documentos PDF y un conjunto de 20 consultas de referencia para su evaluación posterior. Estos valores constituyen el alcance inicial del prototipo, no resultados experimentales obtenidos. La línea base de recuperación, tiempo de consulta y calidad de respuesta será establecida durante las pruebas, permitiendo contrastar el comportamiento del sistema con los criterios de aceptación definidos.

2.2. Necesidad identificada

Los antecedentes examinados permiten identificar tres necesidades tecnológicas principales: mejorar la localización de información distribuida en documentos, controlar la fidelidad de las respuestas generadas y mantener la consulta de información operativa dentro de límites de autorización definidos.

En primer lugar, se requiere un mecanismo capaz de recuperar fragmentos relevantes de manuales técnicos sin depender exclusivamente de coincidencias literales. La investigación de Karpukhin et al. (2020) fundamenta el uso de recuperación densa, mientras que Robertson y Zaragoza (2009) proporcionan las bases de recuperación probabilística mediante BM25. Estas técnicas permiten justificar distintas estrategias de recuperación, aunque su rendimiento en el corpus seleccionado deberá verificarse experimentalmente.

En segundo lugar, es necesario distinguir entre respuestas sustentadas, respuestas incorrectas y consultas para las que no existe evidencia suficiente. Adlakha et al. (2024) identificaron dificultades de fidelidad en modelos generativos que responden preguntas basándose en información proporcionada.

Asimismo, Saad-Falcon et al. (2024) evaluaron sistemas RAG en ocho tareas de conocimiento mediante ARES, considerando la relevancia contextual, la fidelidad de las respuestas y su pertinencia respecto de la consulta.

En tercer lugar, determinadas consultas pueden requerir información actual sobre servicios informáticos que no se encuentra disponible en documentos estáticos. Para ello, se plantea incorporar MCP como mecanismo de integración con herramientas autorizadas de solo lectura, manteniendo la validación de operaciones y parámetros fuera del modelo generativo (Model Context Protocol, 2025; Hasan et al., 2025).

Por consiguiente, se identifica la necesidad de diseñar una aplicación web que integre gestión documental, recuperación de información mediante RAG, generación de respuestas fundamentadas y consulta controlada de herramientas MCP. El sistema deberá operar sobre la infraestructura disponible, compuesta por una máquina virtual Microsoft Azure y un equipo local encargado del procesamiento de ProjectSevera.

2.3. Justificación de la propuesta

La solución propuesta utiliza generación aumentada por recuperación para separar el almacenamiento documental de la generación de respuestas. Esta arquitectura permite incorporar, actualizar y eliminar documentos sin necesidad de reentrenar el modelo generativo cada vez que se modifica el repositorio (Lewis et al., 2020; Y. Gao et al., 2024).

La elección de una arquitectura hexagonal permite desacoplar los casos de uso del sistema respecto de tecnologías específicas de almacenamiento, recuperación y comunicación. Los puertos y adaptadores facilitan mantener el backend Java con Spring Boot y delegar el procesamiento de inteligencia artificial a ProjectSevera mediante un servicio Python (Cockburn, 2005).

Desde la perspectiva de seguridad, el sistema restringirá las operaciones administrativas mediante autenticación y limitará las herramientas MCP a consultas autorizadas de solo lectura. Esta

decisión responde a los riesgos de inyección de instrucciones y uso indebido de herramientas identificados por Greshake et al. (2023), Hasan et al. (2025) y Hou et al. (2026).

La evaluación del prototipo se realizará mediante criterios verificables asociados con recuperación de información, fidelidad de respuestas, tiempos de procesamiento, control de acceso y tolerancia a fallos. Se utilizarán los requerimientos funcionales y no funcionales establecidos en la sección 4 como base para comprobar el cumplimiento de la propuesta.

De esta manera, el proyecto busca establecer una arquitectura técnicamente viable para la asistencia documental mediante RAG y MCP, utilizando recursos informáticos disponibles y un alcance experimental delimitado. Los resultados cuantitativos de funcionamiento serán obtenidos durante la Unidad III, sin atribuir anticipadamente mejoras que todavía no han sido medidas.

3. Objetivos

3.1. Objetivo general

Diseñar una aplicación web de asistencia técnica basada en RAG y MCP que permita consultar documentación PDF mediante lenguaje natural y recibir respuestas sustentadas en fuentes identificables, utilizando un backend Java con arquitectura hexagonal y una infraestructura distribuida entre Azure y ProjectSevera en un equipo local.

3.2. Objetivos específicos

1. Diseñar un módulo administrativo que permita incorporar, listar, almacenar y retirar documentos PDF, manteniendo la correspondencia entre archivos, metadatos e índice de recuperación.
2. Definir el subsistema RAG de ProjectSevera para procesar documentos, recuperar evidencia, generar respuestas citadas e informar cuando no exista sustento suficiente.
3. Definir una integración MCP de solo lectura con herramientas previamente autorizadas y controles de validación en los límites de aplicación.
4. Establecer los componentes, la infraestructura, el diseño de red, la seguridad, la operación y las pruebas necesarias para validar el prototipo en la Unidad III.

4. Requerimientos

4.1. Casos de uso y alcance funcional

El sistema presenta dos actores: Usuario, que consulta la información sin crear una cuenta, y Administrador, que también consulta pero debe autenticarse para gestionar documentos. El caso de uso de herramientas MCP amplía el de consulta IA cuando hace falta obtener un dato operativo. La figura 1 representa estas relaciones; el inicio de sesión se mantiene como caso independiente y condición previa para administrar archivos.

Tabla 1

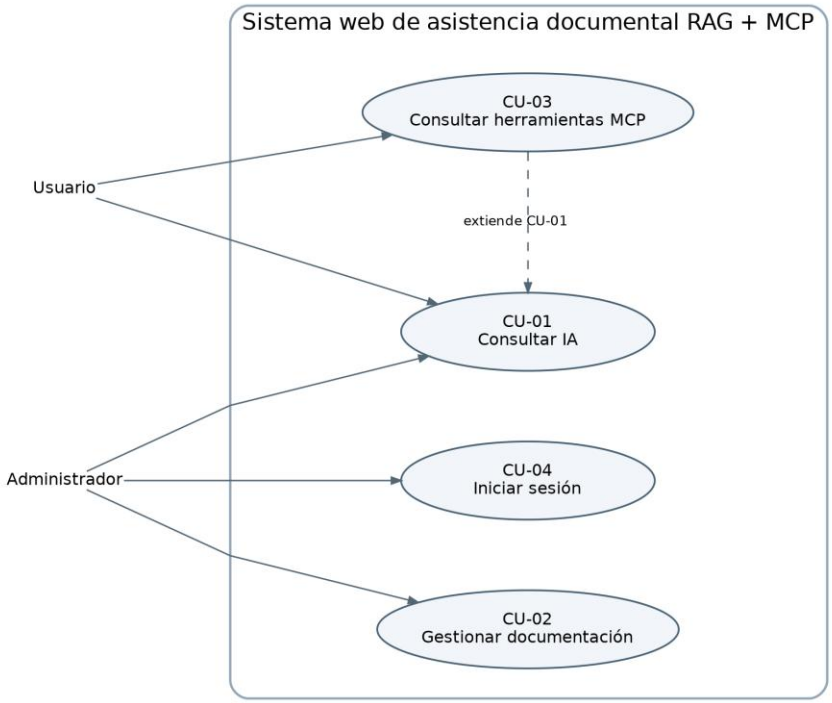
Actores y casos de uso del prototipo

Código	Actor	Caso de uso	Alcance
CU-01	Usuario y administrador	Consultar IA	Pregunta, recuperación RAG, respuesta con fuentes o abstención.
CU-02	Administrador	Gestionar documentación	Cargar, listar, retirar PDF y mantener sincronizado el índice.
CU-03	Usuario y administrador	Consultar herramientas MCP	Extensión de CU-01; consultas permitidas de solo lectura.
CU-04	Administrador	Iniciar sesión	Autenticar o cerrar sesión antes de gestionar documentos.

La Tabla 1 presenta los actores identificados y los cuatro casos de uso definidos para el funcionamiento del prototipo.

Figura 1

Diagrama de casos de uso del sistema RAG + MCP



La Figura 1 fue creada mediante los casos de uso elaborados de manera propia

La separación entre CU-01 y CU-03 no implica crear dos aplicaciones de chat: el usuario sigue interactuando con una única interfaz. CU-03 representa un resultado distinto —lectura de información de una herramienta— que puede activarse dentro de una consulta. La autenticación solo se requiere para CU-02; esta delimitación evita implantar un registro de cuentas innecesario (Model Context Protocol, 2025; Yao et al., 2023).

4.2. Requerimientos funcionales

Tabla 2

Requerimientos funcionales y casos de uso asociados

ID	Capacidad del sistema	CU
RF-01	Permitir consultas en lenguaje natural sin exigir registro del usuario.	CU-01

RF-02	Recuperar fragmentos documentales y generar respuestas con identificación del PDF de origen.	CU-01
RF-03	Informar cuando el material disponible no permita sustentar una respuesta.	CU-01
RF-04	Permitir al administrador autenticado cargar PDF y consultar su listado.	CU-02
RF-05	Eliminar documentos y retirar sus fragmentos del índice RAG, conservando un estado de sincronización.	CU-02
RF-06	Procesar documentos cargados e informar estados pendiente, procesado, error o eliminado.	CU-02
RF-07	Consultar herramientas MCP autorizadas de solo lectura y presentar su resultado.	CU-03
RF-08	Permitir al administrador iniciar y cerrar sesión mediante credenciales.	CU-04

La Tabla 2 establece los ocho requerimientos funcionales del sistema y su correspondencia con los casos de uso previamente definidos.

RF-02 y RF-03 representan capacidades diferentes: generar una respuesta fundamentada y reconocer la insuficiencia de evidencia. Los trabajos sobre fidelidad de respuestas y autoevaluación RAG muestran por qué ambas situaciones deben probarse de manera separada (Adlakha et al., 2024; Asai et al., 2024; T. Gao et al., 2023). El procesamiento documental se documenta como operación funcional, mientras que el tamaño de los archivos, el rendimiento y las restricciones de acceso se establecen mediante RNF.

4.3. Requerimientos no funcionales y atributos de calidad

Los once RNF se organizan según atributos del modelo de calidad de producto ISO/IEC 25010:2023. La selección responde a los riesgos del diseño: seguridad frente a permisos indebidos, fiabilidad frente a caídas de Severa, rendimiento en el entorno real, interoperabilidad Java–Python y

posibilidad de modificar adaptadores. Los valores indicados son umbrales propuestos para pruebas futuras, no resultados existentes (ISO/IEC, 2023; Saad-Falcon et al., 2024).

Tabla 3
Atributos de calidad y criterios de aceptación

ID	Atributo / propiedad	Criterio verificable
RNF-01	Seguridad / autenticidad	Rechazar 10 de 10 solicitudes administrativas sin sesión válida.
RNF-02	Seguridad / resistencia a ataques	Rechazar 10 de 10 llamadas a herramientas MCP no autorizadas.
RNF-03	Seguridad / integridad	Rechazar todos los archivos inválidos y superiores al límite configurado en la batería de prueba.
RNF-04	Fiabilidad / tolerancia a fallos	Informar caída de Severa en ≤10 s, en cinco ensayos, sin fingir respuesta.
RNF-05	Fiabilidad / recuperabilidad	Crear respaldos diarios de PDF y metadata.json; completar una restauración verificable.
RNF-06	Eficiencia / tiempo de respuesta	Al menos 18 de 20 consultas RAG en ≤60 s con un usuario concurrente.
RNF-07	Capacidad de interacción / operabilidad	Realizar consulta y ver fuente en ≤3 acciones principales.
RNF-08	Mantenibilidad / modificabilidad	Sustituir PythonAiAdapter por un adaptador simulado sin cambiar Domain ni casos de uso.
RNF-09	Compatibilidad / interoperabilidad	Superar 10 de 10 intercambios Java–Python acordes con el contrato de API.
RNF-10	Flexibilidad / escalabilidad	Incorporar al menos 20 PDF válidos sin modificar el código fuente.
RNF-11	Adecuación funcional / corrección	Sustentar correctamente en evidencia al menos 16 de 20 respuestas de referencia.

La Tabla 3 presenta los once requerimientos no funcionales, sus atributos de calidad y los criterios verificables que permitirán evaluar su cumplimiento.

La meta RNF-11 se verificará con un conjunto de referencia definido antes de ejecutar las pruebas, para reducir el riesgo de calificar únicamente respuestas favorables. La fidelidad al contexto y

la corrección de la respuesta son dimensiones relacionadas pero distintas; se registrarán ambas cuando proceda (Es et al., 2024; Friel et al., 2024; Saad-Falcon et al., 2024).

5. Diseño de la arquitectura

5.1. Arquitectura lógica hexagonal

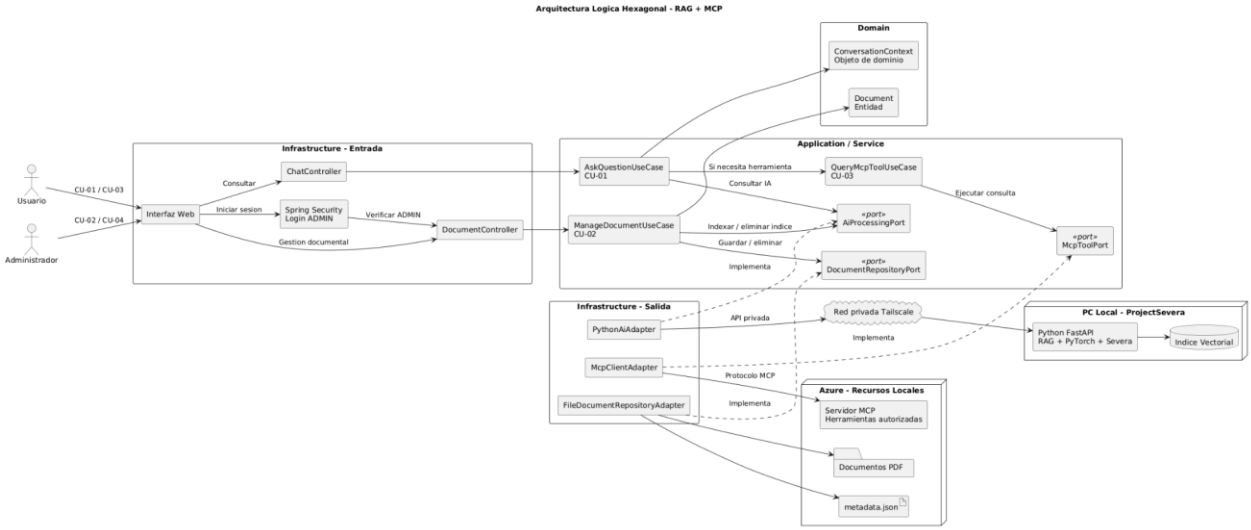
La arquitectura de aplicación se organiza en Domain, Application/Service e Infrastructure. Domain contiene Document y ConversationContext, así como reglas independientes de Spring Boot. Application define AskQuestionUseCase, ManageDocumentUseCase y QueryMcpToolUseCase; también declara AiProcessingPort, DocumentRepositoryPort y McpToolPort. Infrastructure reúne los controladores de entrada y los adaptadores de salida. Esta organización persigue que las decisiones de negocio dependan de contratos estables, no del motor de IA o del formato de almacenamiento (Cockburn, 2005; ISO/IEC, 2023).

ChatController invoca el caso de uso de consulta. DocumentController expone las operaciones del administrador y se somete al control de Spring Security. La cuenta de administración estará configurada en memoria con una contraseña protegida mediante BCrypt, sin gestión de registro público. Esta opción corresponde al tamaño del prototipo; la documentación oficial de Spring Security permite autenticar usuarios desde InMemoryUserDetailsManager sin un repositorio relacional (Spring Security, s. f.).

En la figura 2, las dependencias de las clases de aplicación apuntan a puertos y los adaptadores implementan esos contratos. PythonAiAdapter encapsula la API privada de Severa, FileDocumentRepositoryAdapter administra PDF y metadatos, y McpClientAdapter invoca herramientas permitidas. No se presentan los tres adaptadores como servidores separados: son componentes del backend Java.

Figura 2

Arquitectura lógica hexagonal con puertos, adaptadores y casos de uso



Nota. Elaboración propia.

El diseño evita que la aplicación conozca las librerías internas de ProjectSevera. Una sustitución del modelo o del índice vectorial deberá preservar el contrato del servicio Python y la semántica de las respuestas. Esta decisión se verificará con un adaptador simulado, según RNF-08.

5.2. Arquitectura física y despliegue

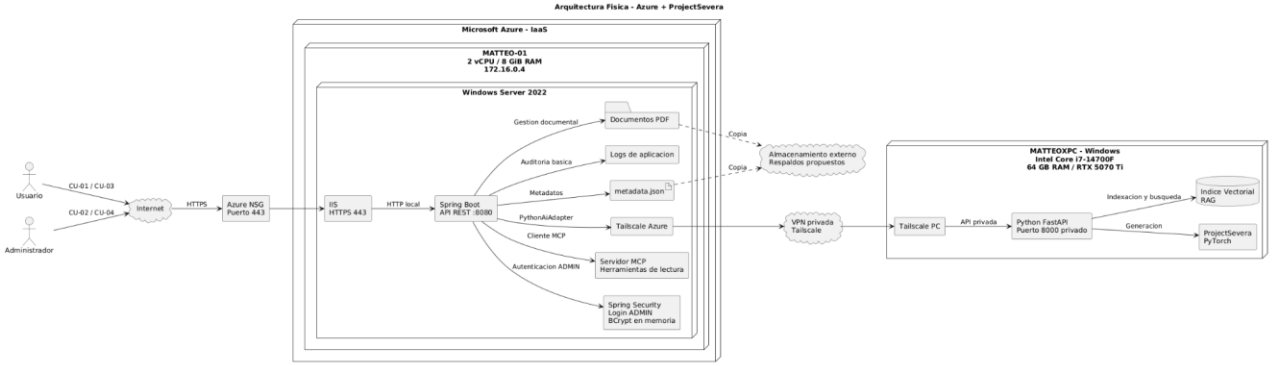
La topología física estará formada por dos nodos: la máquina virtual MATTEO-01 alojada en Azure y la estación MATTEOXPC. El primer nodo expone HTTPS mediante IIS y aloja Spring Boot, Spring Security, el servidor MCP de lectura, los PDF originales y metadata.json. El segundo ejecuta FastAPI, ProjectSevera con PyTorch, los embeddings y el índice vectorial. Una VPN Tailscale proporciona conectividad privada entre ambos equipos. No se utilizan tres servidores físicos, ni una base de datos PostgreSQL, ni Proxmox.

La máquina Azure disponible es una Standard B2as v2 con 2 vCPU y 8 GiB de RAM, sobre Windows Server 2022. El procesamiento principal de inferencia se traslada a MATTEOXPC (Intel Core i7-14700F, 64 GB de RAM y NVIDIA RTX 5070 Ti) para evitar prometer capacidades de GPU en Azure que no existen en la VM. El uso de búsqueda vectorial y la ejecución de modelos en GPU tienen antecedentes

técnicos, pero su rendimiento concreto dependerá de la configuración real de Severa (Johnson et al., 2021; Malkov y Yashunin, 2020; Microsoft, s. f.).

Figura 3

Despliegue físico: máquina virtual Azure y equipo local ProjectSevera



Nota. Elaboración propia.

La figura 3 representa una dependencia operativa que debe quedar explícita: si MATTEOXPC está apagada o la red privada se interrumpe, la web puede seguir publicándose, pero las consultas que requieren inferencia o recuperación no podrán completarse. No se declarará alta disponibilidad de IA en esta versión.

5.3. Componentes principales y decisiones de diseño

Tabla 4

Responsabilidad y ubicación de los componentes principales

Componente	Tecnología / interfaz	Ubicación y función
Interfaz web	HTML, CSS, JavaScript	Azure: chat público y carga administrativa.
API y servicios	Spring Boot, REST	Azure: entrada, casos de uso, validación y coordinación.
Seguridad de aplicación	Spring Security + BCrypt	Azure: cuenta ADMIN en memoria; sin registro público.
Almacenamiento base	FileDocumentRepositoryAdapter	Azure: documentos PDF y metadata.json.
Procesamiento RAG	FastAPI + PyTorch + Severa	MATTEOXPC: extraer, segmentar, vectorizar, recuperar y generar.
Índice vectorial	Motor persistente por verificar	MATTEOXPC: fragmentos y embeddings indexados.

Interoperabilidad	McpClientAdapter + servidor MCP	Azure: herramientas previamente autorizadas.
Conexión privada	Tailscale	Azure ↔ PC: solicitudes API protegidas.

La Tabla 4 describe los componentes principales de la arquitectura, las tecnologías seleccionadas y las responsabilidades asignadas a cada elemento.

La ingesta documental seguirá una secuencia controlada: el administrador remite un PDF, el backend verifica su extensión, firma real, tamaño y permisos; después almacena el original y registra un identificador único. El servicio Python extrae el texto, divide el documento en fragmentos, calcula representaciones vectoriales y actualiza el índice. Se conservará una relación entre identificador de documento, versión y fragmentos para permitir la eliminación posterior. La granularidad de la recuperación afecta los resultados; el estudio de Chen et al. muestra que elegir la unidad de indexación es una decisión experimental y no una constante universal (Barnett et al., 2024; Chen et al., 2024).

En el primer prototipo se propone recuperar los fragmentos más próximos a la consulta mediante un índice vectorial persistente. La selección exacta del modelo de embeddings y del motor de índice se documentará con su versión y se medirá con consultas de referencia. La literatura sobre DPR, ColBERT, HNSW y FAISS permite justificar alternativas, sin afirmar que todas esas técnicas estén implementadas simultáneamente (Johnson et al., 2021; Karpukhin et al., 2020; Khattab y Zaharia, 2020; Malkov y Yashunin, 2020).

La coincidencia léxica BM25 se considera una alternativa de mejora para consultas con códigos de error y nombres exactos. El reranking con modelos adicionales, la recuperación jerárquica y GraphRAG no forman parte de los componentes comprometidos: se mencionan para evaluar límites y explicar por qué un índice vectorial sencillo es suficiente como línea base de la Unidad III (Edge et al., 2024; Izacard y Grave, 2021; Nogueira et al., 2020; Robertson y Zaragoza, 2009).

Para la generación, ProjectSevera recibirá una estructura que incluya la consulta original, fragmentos recuperados y metadatos de procedencia. La respuesta deberá identificar el documento

utilizado; si no existen fragmentos adecuados, el sistema comunicará que la evidencia es insuficiente. Los modelos generativos pueden producir texto correcto en apariencia que no esté respaldado, por lo que mostrar una cita no basta: también habrá que verificar que esa cita realmente sustente la afirmación (Adlakha et al., 2024; Ji et al., 2023; T. Gao et al., 2023).

El módulo MCP tendrá un conjunto cerrado de herramientas de lectura. Se proponen getDocumentIndexStatus para consultar el estado general de indexación y getServiceStatus para revisar el estado de un servicio previamente permitido en el entorno de laboratorio. El cliente enviará parámetros de esquema limitado y el servidor validará la operación. Los estudios sobre uso de herramientas muestran que conviene evaluar tanto la selección de la herramienta como la corrección de sus argumentos (Li et al., 2023; Qin et al., 2024; Schick et al., 2023; Yao et al., 2023).

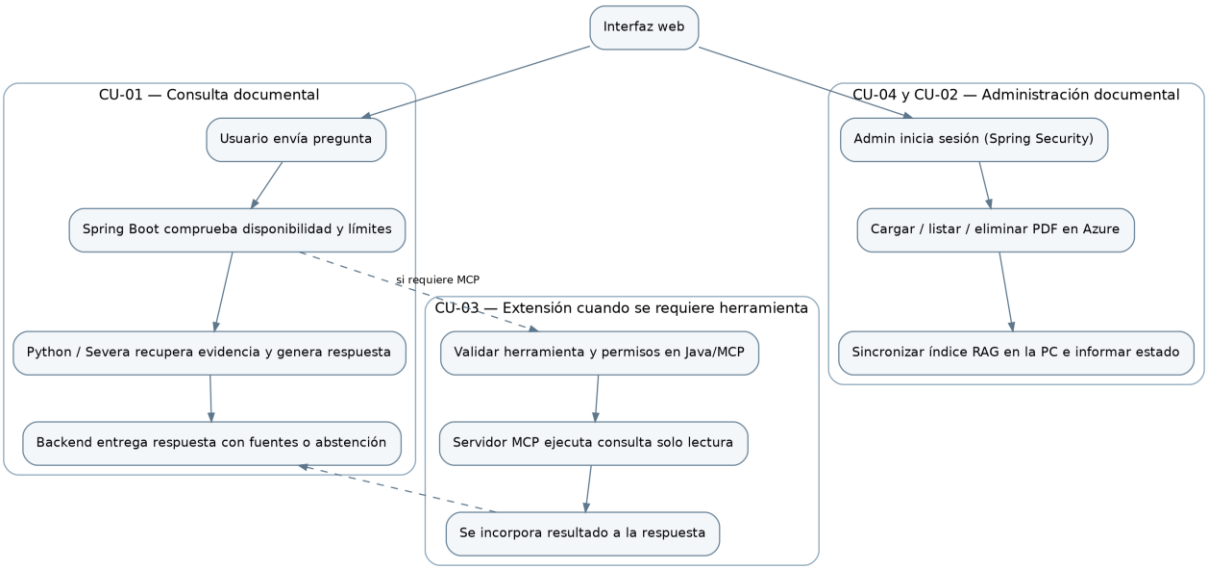
5.4. Flujo de información

El flujo de CU-01 comienza con una pregunta desde la web. Spring Boot comprueba los límites de la solicitud y llama a AiProcessingPort. Python consulta el índice local, recupera evidencia y genera un borrador fundamentado. Si la solicitud requiere una herramienta, el backend solicita CU-03, valida permisos y parámetros, recibe el dato operativo y lo incorpora a la respuesta final; el resultado de la herramienta no se interpreta como una instrucción de administración. Los mecanismos de recuperación activa y el protocolo de herramientas muestran la necesidad de mantener separadas las observaciones del sistema y las decisiones de ejecución (Hou et al., 2026; Jiang et al., 2023; Model Context Protocol, 2025).

En CU-04 el administrador inicia sesión antes de ejecutar CU-02. Después de una carga válida, Java guarda el PDF y su registro en metadata.json; solicita la indexación a Severa y actualiza el estado según la respuesta del servicio. Cuando se elimina un PDF, la operación debe retirar también los fragmentos asociados. Si la PC no está disponible, el sistema mantendrá la operación pendiente para reintento y no informará que se indexó un documento que todavía no fue procesado.

Figura 4

Flujo de información: consulta, extensión MCP y administración documental



Nota. Elaboración propia.

6. Diseño de infraestructura

6.1. Servidores, equipos y capacidades

Tabla 5

Inventario físico y virtual utilizado como base del diseño

Nodo	Recursos conocidos	Responsabilidad
MATTEO-01 (Azure)	Windows Server 2022; Standard B2as v2; 2 vCPU; 8 GiB RAM	Publicación web, backend, autenticación, MCP, PDF y metadatos.
MATTEOXPC (local)	Intel Core i7-14700F; 64 GB RAM; NVIDIA RTX 5070 Ti	Inferencia de Severa, procesamiento PDF e índice vectorial.

La Tabla 5 presenta el inventario de recursos físicos y virtuales disponibles, considerando las características de MATTEO-01 y MATTEOXPC.

Los valores corresponden a equipos disponibles y deben distinguirse de recursos todavía no medidos, como el espacio exacto de los discos de Azure, el volumen final del corpus y la memoria de GPU utilizada durante una consulta. El diseño evita asignar una latencia garantizada sin pruebas. El uso de recursos GPU para cálculos vectoriales está documentado, pero no permite trasladar directamente

resultados de un benchmark a esta configuración particular (Johnson et al., 2021; Muennighoff et al., 2023).

6.2. Sistemas operativos y virtualización

Azure proporciona una máquina virtual Windows Server 2022 bajo el modelo IaaS. La virtualización del nodo ya es un servicio de Azure; no se instalarán Proxmox, Hyper-V ni contenedores de orquestación por obligación. MATTEOXPC utiliza Windows y ejecutará el entorno Python de ProjectSevera. Esta separación permite desplegar el backend de forma independiente del modelo de IA, aunque introduce la dependencia de conectividad entre ubicaciones (Microsoft, s. f.).

Los componentes de Azure se ejecutarán como servicios o procesos administrados en el propio Windows Server: IIS publicará HTTPS y redirigirá a Spring Boot escuchando en loopback. El servidor MCP se comunicará con el backend a través de un transporte local autorizado. En la PC, FastAPI atenderá exclusivamente a la conexión privada de Tailscale. Se establecerán versiones estables de Java, Python y librerías antes de las pruebas finales.

6.3. Almacenamiento, sincronización y capacidad

El repositorio original en Azure consistirá en una carpeta controlada con archivos PDF y un archivo metadata.json. Cada registro incluirá identificador, nombre, huella SHA-256, fecha de carga, versión, estado de procesamiento y fecha de última indexación. MATTEOXPC mantendrá una copia de trabajo de los fragmentos procesados, embeddings y un índice persistente; este índice es regenerable a partir de los originales siempre que se conserven los PDF y la configuración de procesamiento.

La sincronización requiere dos garantías prácticas. La primera consiste en evitar que un documento borrado continúe apareciendo en las búsquedas; la segunda es permitir que una carga fallida quede identificada para reprocesamiento. Se utilizarán estados pendiente, procesado, error y eliminado, con confirmación de operación entre Java y Python. La elección de un identificador estable facilita localizar los fragmentos afectados por cada cambio. Los trabajos sobre recuperación documental

y fragmentación advierten que la extracción y el tratamiento de contexto pueden introducir errores antes de la etapa de generación (Barnett et al., 2024; Chen et al., 2024; Feng et al., 2021).

Para dimensionar el almacenamiento durante las pruebas se registrará el tamaño real de veinte PDF de muestra, la expansión del texto extraído y el espacio ocupado por el índice. No se supone un tamaño empresarial de almacenamiento. Se definirá un límite de archivo según el espacio medido en Azure y el tiempo de procesamiento observado en MATTEOXPC.

6.4. Servicios requeridos

Tabla 6
Servicios y exposición prevista

Servicio	Equipo	Punto de acceso
IIS / certificado TLS	Azure	TCP 443 público.
Spring Boot / Spring Security	Azure	127.0.0.1:8080; solo interno.
Servidor MCP solo lectura	Azure	Transporte local permitido por Java.
Repositorio PDF + metadata.json	Azure	Acceso mediante permisos del sistema de archivos.
Tailscale	Azure y PC	Red privada entre equipos autorizados.
FastAPI + Severa + índice	PC local	API privada propuesta TCP 8000.
Registro y respaldos	Azure y destino externo	Almacenamiento controlado, sin acceso público.

La Tabla 6 identifica los servicios requeridos para el funcionamiento del sistema, indicando su ubicación y los mecanismos de acceso previstos.

6.5. Restricciones y capacidad prevista

El prototipo atenderá inicialmente un usuario concurrente durante las pruebas de rendimiento. La máquina Azure no hará inferencia GPU y el equipo local no funcionará como servidor empresarial permanente. Si Severa no está disponible, la web informará la condición. Esta decisión limita la viabilidad operativa a horarios de demostración y requiere que el plan de pruebas contemple fallos deliberados del enlace.

7. Diseño de red

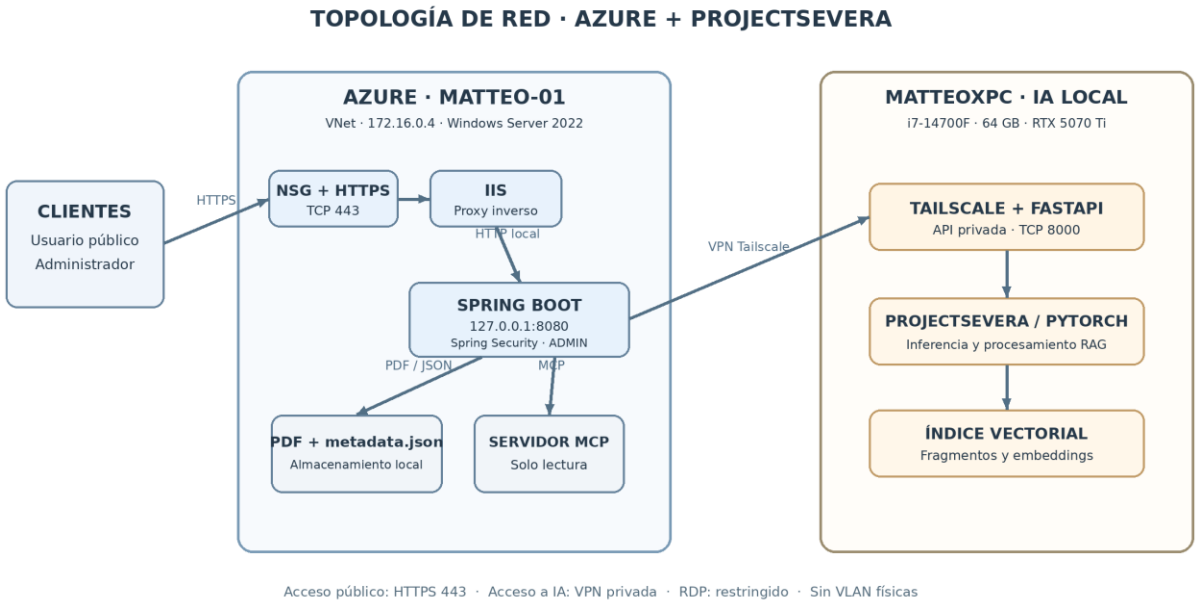
7.1. Topología y segmentación

Los clientes se conectarán desde internet al servidor HTTPS en Azure, protegido por un Network Security Group (NSG) y Windows Firewall. La comunicación interna entre IIS y Spring Boot permanecerá en loopback. La aplicación Java usará Tailscale para alcanzar la API Python de MATTEOXPC sin publicar directamente el puerto de inferencia en internet. La figura 5 representa estas tres zonas lógicas: acceso público, servicios internos de Azure y procesamiento privado de IA.

La segmentación propuesta se basa en las funciones efectivamente desplegadas, no en una topología de switches físicos. Los recursos de Azure Virtual Network permiten definir subredes y reglas de filtrado de tráfico, mientras que las políticas de acceso de Tailscale restringen qué equipos pueden utilizar un servicio privado (Microsoft, s. f.; Tailscale, s. f.).

Figura 5

Topología de red híbrida: Azure, VPN y MATTEOXPC



Nota. Elaboración propia.

7.2. Direccionamiento IP

Tabla 7

Direcciones y puertos del diseño de red

Elemento	Dirección o puerto	Condición
VM Azure	172.16.0.4	IP privada observada.
VNet y subred	vnet-chilecentral-1 / snet-chilecentral-1	Nombres observados en Azure.
IP pública	Asignada por Azure	Confirmar asignación y vigencia.
IIS	TCP 443	Entrada pública HTTPS.
Spring Boot	127.0.0.1:8080	Solo loopback de la VM.
Servidor MCP	Transporte local	No publicar puerto.
FastAPI Severa	IP Tailscale:8000	Dirección por obtener al configurar VPN.
RDP	TCP 3389	Administración restringida.

La Tabla 7 presenta las direcciones IP, los puertos de comunicación y las condiciones de acceso correspondientes a los componentes de la infraestructura.

El rango CIDR de la subred no se deduce de la IP 172.16.0.4; deberá obtenerse directamente de la consola de Azure. Tampoco se atribuye una IP de Tailscale antes de registrar ambos nodos. Estas reservas impiden declarar direcciones inventadas o subredes que podrían solaparse.

7.3. VLAN y aislamiento equivalente

No se implementarán VLAN físicas IEEE 802.1Q. El diseño contiene una VM alojada en una VNet de Azure y una PC local conectada mediante VPN. Los servicios se aislarán con NSG, firewalls de host, interfaces de loopback y políticas de la red privada. Este mecanismo no equivale a configurar VLAN físicas, pero satisface la necesidad de separar las zonas pública, interna y de IA sin simular infraestructura inexistente (Microsoft, s. f.).

7.4. Firewall y acceso remoto

Tabla 8

Políticas de comunicación y puntos de aplicación

ID	Origen	Destino / servicio	Acción
FW-01	Internet	IIS Azure TCP 443	Permitir
FW-02	Internet	Spring Boot 8080 / MCP	No publicar / denegar
FW-03	Internet	FastAPI PC TCP 8000	No publicar / denegar
FW-04	Tailscale Azure	API PC TCP 8000	Permitir solo equipo autorizado

FW-05	Administrador autorizado	RDP Azure TCP 3389	Restringir por VPN o IP autorizada
FW-06	Cualquier otro origen	Servicios internos	Denegar entrada por defecto

La Tabla 8 establece las políticas de comunicación propuestas, especificando los orígenes, destinos y acciones permitidas o restringidas.

FW-01, FW-02 y FW-05 se materializarán mediante Azure NSG y Windows Firewall según el punto de entrada. FW-03 y FW-04 requieren firewall local y políticas de Tailscale en MATTEOXPC, porque una regla NSG de Azure no controla directamente el tráfico entrante a una PC doméstica. FW-06 expresa la política general de mínimos permisos y debe comprobarse con pruebas de conectividad, no solo con una captura de configuración.

El acceso administrativo RDP no se abrirá indiscriminadamente a internet. Antes de cerrar una regla temporal existente, se verificará que la conexión alternativa por VPN funcione para evitar perder el acceso al servidor. Ningún secreto de Tailscale, cookie de sesión o credencial de administrador deberá incluirse en capturas públicas (NIST, 2020; Tailscale, s. f.).

8. Diseño de seguridad

8.1. Identificación de amenazas y valoración de riesgos

La valoración preliminar sigue el criterio de identificación, análisis y tratamiento de riesgos asociado a ISO/IEC 27005:2022. Se aplica una matriz simple de probabilidad P e impacto I con valores de 1 a 3. El riesgo se calcula como P × I: bajo para 1–2, medio para 3–4 y alto para 6–9. Son estimaciones previas a la implementación y deberán revisarse con resultados de pruebas (ISO/IEC, 2022; NIST, 2020).

La integración de un generador con documentos y herramientas introduce una frontera de confianza adicional. Un PDF puede contener instrucciones que intenten alterar el comportamiento del asistente y una definición de herramienta puede inducir llamadas indebidas. Tanto las investigaciones de inyección indirecta como los estudios recientes sobre MCP justifican que la autorización no se

entregue al modelo generativo (Debenedetti et al., 2024; Greshake et al., 2023; Hasan et al., 2025; Hou et al., 2026; Zhan et al., 2024).

Tabla 9
Matriz de amenazas y riesgo inicial

ID	Amenaza / consecuencia	P	I	P×I / nivel
R-01	Acceso indebido a administración	2	3	6 / Alto
R-02	Prompt injection desde PDF recuperado	2	3	6 / Alto
R-03	Invocación MCP sin autorización	2	3	6 / Alto
R-04	Carga de archivo inválido o malicioso	2	2	4 / Medio
R-05	Exposición pública de servicios internos	2	3	6 / Alto
R-06	Pérdida o corrupción de PDF y JSON	2	2	4 / Medio
R-07	Caída de MATTEOXPC o del enlace VPN	3	2	6 / Alto

La Tabla 9 presenta los riesgos identificados y su valoración inicial mediante los criterios de probabilidad e impacto..

8.2. Controles y estrategias

Tabla 10
Estrategias de seguridad y riesgos tratados

Código	Control propuesto	Riesgos
E-01	Spring Security, ADMIN en memoria, BCrypt y comprobación de rol.	R-01
E-02	PDF validado, límites de carga, tratamiento de contenido como datos y abstención.	R-02 y R-04
E-03	Herramientas MCP solo lectura, esquema de parámetros y allowlist en Java y servidor.	R-02 y R-03
E-04	HTTPS, NSG, firewalls, Tailscale y control de tasa de consultas públicas.	R-05
E-05	Respaldo externo, registros	R-06 y R-07

	mínimos y manejo de indisponibilidad.

La Tabla 10 relaciona las estrategias de seguridad propuestas con los riesgos que buscan mitigar dentro de la arquitectura.

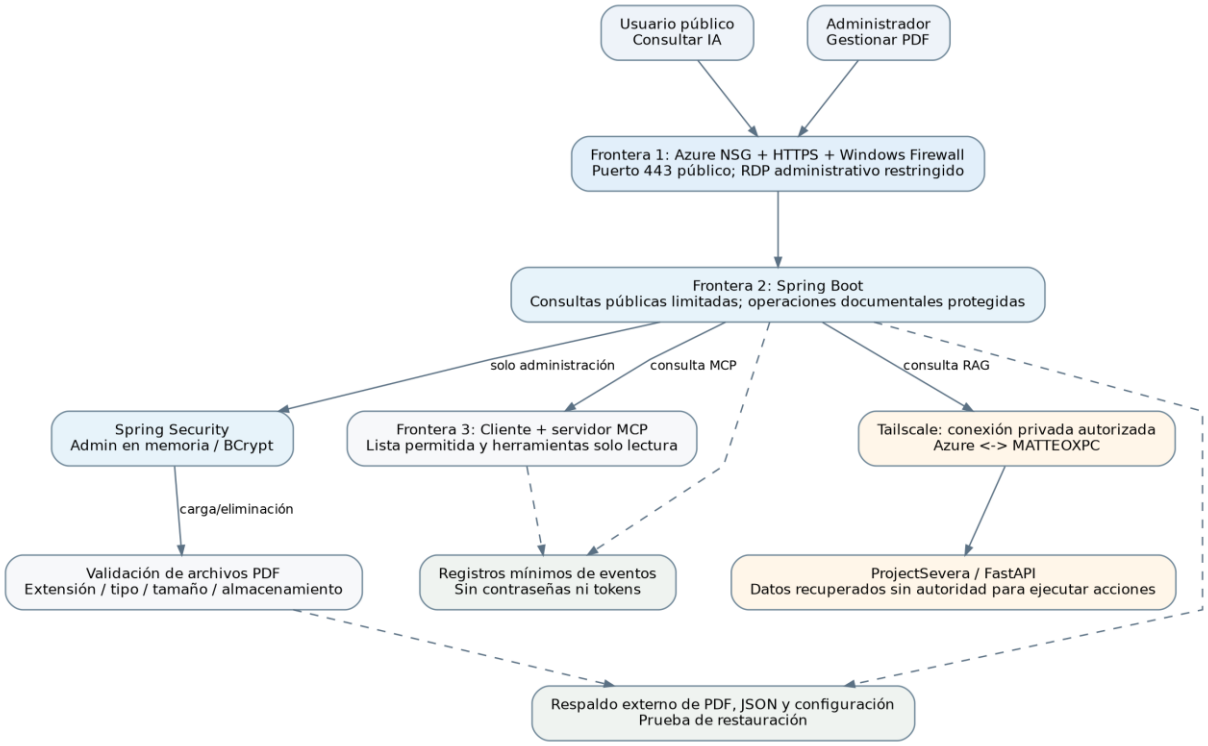
La defensa frente a prompt injection no se basa en una frase del prompt que indique al modelo que ignore ataques. Se divide la información documental de las instrucciones privilegiadas, se comprueban parámetros en código y se deniegan herramientas que no estén en la lista permitida. Los resultados de los estudios AgentDojo e InjecAgent sustentan la necesidad de diseñar pruebas adversariales específicas; no autorizan a declarar que los ataques quedan eliminados (Debenedetti et al., 2024; Greshake et al., 2023; Zhan et al., 2024).

8.3. Diagrama y fronteras de seguridad

La figura 6 organiza controles en frontera de red, de aplicación, de herramientas y de recuperación documental. Spring Boot gestiona la autorización del administrador y las políticas para invocar herramientas; ProjectSevera devuelve resultados pero no tiene facultad para conceder permisos en Azure.

Figura 6

Fronteras de seguridad entre internet, Azure, MCP y ProjectSevera



Nota. Elaboración propia.

8.4. Gestión de accesos

Tabla 11

Identidades y permisos mínimos

Identidad	Acceso autorizado	Restricción
Usuario público	Consultar RAG y solicitar datos MCP permitidos	Sin cargar/eliminar archivos ni usar herramientas arbitrarias.
Administrador	Consultar; autenticarse; cargar/listar/eliminar PDF	Una cuenta ADMIN; credenciales fuera del repositorio.
Backend Java	Leer/escribir repositorio documental; consultar MCP y API privada	Solo puertos y recursos necesarios.
Servicio Python	Procesar documentos y devolver resultados	Sin permiso sobre usuarios o administración Azure.

La Tabla 11 establece las identidades del sistema y los permisos mínimos necesarios para ejecutar las operaciones autorizadas.

La contraseña administrativa se almacenará codificada mediante BCrypt en configuración protegida, no en un archivo público ni en texto plano. La autenticación en memoria reduce el alcance,

pero también requiere controlar la distribución de secretos, la expiración de sesión y la protección contra solicitudes administrativas no autorizadas. La aplicación deberá validar permisos del lado servidor y no confiar únicamente en ocultar un botón del frontend (NIST, 2020; Spring Security, s. f.).

8.5. Respaldo y recuperación

Se propone respaldo diario de los PDF originales y metadata.json hacia un destino independiente de la VM, con conservación inicial de siete copias. El respaldo de ProjectSevera que existe en Google Drive no equivale a un respaldo completo de la aplicación Azure: cubre solamente los archivos que efectivamente se hayan copiado. El índice vectorial de MATTEOXPC se respaldará o podrá reconstruirse utilizando documentos, identificadores y configuración conservados.

El objetivo de pérdida máxima de datos para documentos originales será de hasta 24 horas si la copia diaria se realiza correctamente; el tiempo de recuperación no se presenta como garantía debido a la ausencia de infraestructura redundante. La Unidad III deberá ejecutar una restauración completa de un conjunto de prueba y anotar su duración. La protección de respaldos y la verificación periódica se relacionan con controles de contingencia como CP-9 de NIST SP 800-53 (ISO/IEC, 2022; NIST, 2020).

8.6. Monitoreo y auditoría básica

No se instalará un SIEM empresarial. Se utilizarán las métricas disponibles de Azure, el Visor de eventos de Windows y registros producidos por Spring Boot, FastAPI y MCP. Se anotarán fecha y hora, tipo de evento, resultado, identificador de operación y usuario cuando aplique. Los errores de indexación y las invocaciones MCP tendrán un código que facilite rastrear la solicitud entre los dos nodos.

Se excluirán contraseñas, tokens, cookies y el texto íntegro de documentos sensibles de los registros. Los eventos mínimos serán inicio y fallo de sesión administrativa, carga y retiro de PDF, error de indexación, consulta MCP autorizada o rechazada y caída de la API Severa. La incorporación de

auditoría básica responde a la necesidad de reproducir fallos durante las pruebas y a controles generales de registro y supervisión (Hasan et al., 2025; NIST, 2020).

9. Diseño operativo

9.1. Roles y responsabilidades

El equipo puede distribuir las funciones entre cuatro responsabilidades: administración documental, backend, IA e infraestructura. Un integrante puede asumir más de una función. La Tabla 12 fija quién ejecuta y quién aprueba las actividades críticas mediante una matriz RACI reducida; los nombres de los responsables se establecerán internamente al organizar la implementación.

Tabla 12
RACI simplificada para actividades operativas

Actividad	Admin.	Backend	IA	Infra.
Alta y eliminación de PDF	A/R	C	C	I
Cambio de endpoints o adaptadores	I	A/R	C	C
Actualización del modelo / índice	I	C	A/R	C
Firewall, VPN, acceso RDP	I	C	C	A/R
Respaldo y restauración	C	C	C	A/R
Revisión conjunta de pruebas	I	A/R	R	C

Nota. Elaboración propia.

A indica responsable último, R ejecución, C consulta e I información. En la última fila se asignan dos ejecutores, pero una sola responsabilidad final. En un equipo pequeño es preferible dejar explícitas estas decisiones que asumir roles empresariales inexistentes.

9.2. Operación ordinaria

- Comprobar que la VM Azure y MATTEOXPC están encendidas y que Tailscale indica una conexión autorizada entre ambos equipos.

- Verificar IIS, Spring Boot, MCP local y FastAPI; realizar una consulta sencilla con documento de referencia.
- Permitir al administrador cargar o eliminar PDF únicamente tras una sesión autenticada.
- Revisar las operaciones pendientes de indexación y reintentar las que hayan fallado.
- Mantener el acceso web únicamente durante las ventanas de prueba previstas y comunicar cortes del servicio.

9.3. Mantenimiento preventivo

Se propone una revisión semanal de espacio disponible, integridad de PDF, sincronización con el índice, registros, versión de dependencias y estado de respaldos. Toda actualización del modelo, índice o adaptador se acompañará de una prueba documental con evidencia y de una llamada MCP permitida. Los estudios sobre puntos de fallo RAG y riesgos de mantenimiento en MCP muestran que la operación requiere vigilar tanto datos como herramientas, no solo si el proceso sigue encendido (Barnett et al., 2024; Hasan et al., 2025; Hou et al., 2026).

9.4. Gestión de incidentes

Tabla 13
Respuesta inicial prevista ante fallos del prototipo

Incidente	Diagnóstico	Acción y evidencia
Spring Boot no responde	Revisar proceso, puerto y logs	Reiniciar y ejecutar consulta controlada.
Severa no disponible	Comprobar PC, FastAPI y VPN	Informar caída y recuperar enlace.
Índice desactualizado	Comparar PDF, metadata.json e ID indexado	Reprocesar documento afectado.
Archivo perdido	Identificar versión y último respaldo	Restaurar y verificar lectura e indexación.
Herramienta MCP denegada	Verificar lista, permiso y argumentos	Mantener rechazo hasta corrección autorizada.

La Tabla 13 presenta las respuestas operativas previstas ante los principales fallos que podrían afectar la disponibilidad y el funcionamiento del prototipo.

Un incidente quedará documentado por fecha, componente, síntomas, medida tomada y prueba posterior. La operación nunca debe relajar una regla de acceso de forma permanente solo para

conseguir que una demostración funcione. El equipo conservará los registros necesarios para explicar su intervención durante la sustentación (NIST, 2020).

10. Plan de implementación

10.1. Actividades, recursos y dependencias

Tabla 14

Cronograma propuesto para la Unidad III

Fase	Fechas	Actividades	Depende de
F1. Preparación	16–22 nov. 2026	Revisar Azure, HTTPS, VPN, FastAPI, repositorio.	Ambos equipos disponibles.
F2. Núcleo funcional	23–29 nov. 2026	PDF/JSON, carga, indexación, CU-01 y citas.	F1 y API Python.
F3. Integración	30 nov.–3 dic. 2026	MCP, ADMIN, controles, logs y copias.	F2 y contratos definidos.
F4. Validación	4–6 dic. 2026	Pruebas RF/RNF, seguridad, restauración, evidencias.	Integración estable.

La Tabla 14 establece el cronograma de implementación correspondiente a la Unidad III, considerando las actividades, recursos y dependencias del proyecto.

La ejecución será incremental para reducir el riesgo de integrar demasiados componentes a la vez. Primero se verificará el contrato Java–Python, después la carga de documentos y finalmente el acceso a herramientas. Una prueba de consulta con documento fijo es requisito de entrada para desarrollar el resto del flujo. El diseño modular de recuperación y uso de herramientas permite comprobar cada parte por separado antes de ejecutar la prueba integral (Hou et al., 2026; Y. Gao et al., 2024).

10.2. Recursos y criterios de cierre

Los recursos humanos corresponden al equipo académico distribuido entre backend, IA e infraestructura. Los recursos tecnológicos son la VM MATTEO-01, MATTEOXPC, las herramientas de programación, un corpus de al menos veinte PDF de prueba y una conexión Tailscale. Se documentarán versiones de Java, Python, el modelo de Severa y dependencias. La entrega de la Unidad III se

considerará demostrable solo cuando existan evidencias de CU-01 a CU-04 y de las pruebas relacionadas con RF y RNF, sin confundir diseño con implementación.

11. Plan de pruebas

11.1. Pruebas funcionales

Cada ejecución conservará identificador, fecha, versión de software, entrada, resultado observado, evidencia y estado. Las pruebas pueden ejecutarse manualmente con interfaz, solicitudes HTTP y revisión de logs, pero todas requieren criterio explícito de aceptación. El enfoque de pruebas conductuales propone evaluar también variaciones y casos negativos, no únicamente ejemplos donde el sistema funciona como se esperaba (Ribeiro et al., 2020; Saad-Falcon et al., 2024).

Tabla 15

Pruebas funcionales previstas y trazabilidad

Prueba	RF	Entrada / procedimiento	Aceptación
PF-01	RF-01	Enviar pregunta sin iniciar sesión.	Consulta aceptada.
PF-02	RF-02	Preguntar dato localizado en PDF y contrastar cita.	Respuesta sustentada y fuente correcta.
PF-03	RF-03	Consultar tema ausente del corpus.	Informa insuficiencia sin inventar procedimiento.
PF-04	RF-04	Iniciar sesión, cargar y listar PDF.	Archivo conservado y visible.
PF-05	RF-05	Eliminar PDF indexado y repetir búsqueda.	No se recuperan fragmentos del archivo eliminado.
PF-06	RF-06	Cargar documento y consultar estado de proceso.	Estado final procesado o error identificable.
PF-07	RF-07	Solicitar herramienta MCP permitida.	Devuelve dato de lectura, sin cambios en servidor.
PF-08	RF-08	Iniciar sesión, cerrar sesión, repetir acceso.	Ingreso válido; sesión cerrada; acceso protegido.

La Tabla 15 presenta las pruebas funcionales propuestas, vinculando cada procedimiento con los requerimientos correspondientes y sus criterios de aceptación.

11.2. Pruebas de atributos de calidad

Tabla 16
Pruebas no funcionales con criterio medible

Prueba	RNF	Procedimiento / criterio
PN-01	RNF-01	Diez accesos administrativos sin sesión: 10 rechazados.
PN-02	RNF-02	Diez herramientas fuera de lista: 10 rechazadas.
PN-03	RNF-03	Archivos inválidos o mayores al límite: todos rechazados.
PN-04	RNF-04	Cinco caídas de Severa: aviso en ≤10 s.
PN-05	RNF-05	Restaurar PDF/JSON y verificar integridad.
PN-06	RNF-06	Veinte consultas: ≥18 completadas en ≤60 s.
PN-07	RNF-07	Realizar consulta y ver fuente en ≤3 acciones.
PN-08	RNF-08	Sustituir adaptador IA sin editar dominio/casos de uso.
PN-09	RNF-09	Diez respuestas API: 10 conformes al contrato.
PN-10	RNF-10	Incorporar veinte PDF sin modificar código.
PN-11	RNF-11	Veinte preguntas conocidas: ≥16 respuestas sustentadas.

La Tabla 16 establece las pruebas no funcionales destinadas a verificar los atributos de calidad mediante criterios de aceptación medibles.

La métrica de latencia se medirá desde que el backend recibe una consulta válida hasta que entrega su respuesta completa. Se anotarán duración total y tiempos parciales de búsqueda, generación y red cuando estén disponibles. Las métricas de fidelidad y relevancia estudiadas por RAGAS, ARES y RAGBench servirán para complementar la evaluación manual, pero no sustituirán las respuestas de referencia definidas por el equipo (Adlakha et al., 2024; Es et al., 2024; Friel et al., 2024; Saad-Falcon et al., 2024).

11.3. Evaluación documental, seguridad e integración

El conjunto de veinte preguntas incluirá cinco consultas directas, cinco con códigos o nombres exactos, cinco ambiguas o repartidas entre documentos y cinco fuera del corpus. Para cada una se conservarán la respuesta esperada, el documento fuente, fragmentos relevantes y un indicador de abstención cuando corresponda. La comparación de resultados no presupondrá que la recuperación vectorial supera a BM25 en todo tipo de consulta; se registrará el comportamiento observado y, si se dispone del tiempo, se podrá contrastar una línea base léxica sin convertirla en un componente productivo comprometido (Karpukhin et al., 2020; Muennighoff et al., 2023; Nogueira et al., 2020; Robertson y Zaragoza, 2009).

Las pruebas de prompt injection se realizarán con PDF de laboratorio que contengan texto formulado como una supuesta orden privilegiada. Se observará si esa instrucción altera la respuesta, solicita herramientas no autorizadas o consigue modificar permisos. El resultado deseado será que las políticas del backend y del servidor MCP denieguen cualquier operación prohibida, incluso si el modelo genera una petición incorrecta. Se utilizarán escenarios inspirados en estudios de inyección indirecta, AgentDojo e InjecAgent, sin afirmar que diez pruebas agoten todas las variantes de ataque (Debenedetti et al., 2024; Greshake et al., 2023; Zhan et al., 2024).

Se probarán además la caída deliberada del servicio Python, la pérdida temporal de Tailscale y un PDF cuya indexación falle. En cada caso la aplicación deberá mostrar un estado consistente. La sincronización entre metadata.json y el índice será verificada después de restaurar, eliminar y volver a cargar documentos. Los resultados se presentarán en la Unidad III con capturas, logs y versión exacta del modelo.

12. Análisis de factibilidad

12.1. Factibilidad técnica

Se considera plausible construir el prototipo porque el equipo dispone de un servidor Azure para publicación y un equipo local para procesamiento de IA. Spring Boot puede operar con los recursos de la VM para el alcance de laboratorio, mientras la RTX 5070 Ti se reserva para las cargas de PyTorch y del índice. No obstante, la disponibilidad de hardware no demuestra por sí sola que cada RNF se cumpla: las mediciones deberán comprobar la velocidad del modelo, el tamaño del corpus y el desempeño de la comunicación Java–Python (ISO/IEC, 2023; Johnson et al., 2021; Muennighoff et al., 2023).

La principal incertidumbre técnica es la integración real de ProjectSevera con una API estable, incluida la devolución de fuentes y el comportamiento ante evidencia insuficiente. La elección del modelo generativo, del embedding y del índice persistente deberá registrarse sin alterar el contrato expuesto por AiProcessingPort. Se descartan compromisos de alta disponibilidad, múltiples GPU, clústeres y voz por no ser necesarios para validar los cuatro casos de uso.

12.2. Factibilidad económica

El cálculo distingue desembolso efectivo y valoración económica del recurso. La PC y la máquina virtual ya están disponibles, de modo que no se presupone una compra de hardware. Sin embargo, el consumo de créditos de Azure, electricidad y tiempo de trabajo tienen un costo de oportunidad. Las cantidades siguientes son escenarios ilustrativos, no facturas certificadas ni una cotización vinculante.

Tabla 17
Hipótesis de presupuesto y costo mensual estimado

Concepto	Supuesto	Monto mensual
Azure IaaS	US$100/mes × S/3,42 (tipo supuesto)	S/342,00
Electricidad PC: escenario medio	0,9 kW × 720 h × S/0,80/kWh	S/518,40
Mantenimiento equivalente	8 h × S/15,625/h	S/125,00
Total económico orientativo	Infraestructura + energía + mantenimiento	S/985,40
Costo sin imputar trabajo	Azure + electricidad	S/860,40

La Tabla 17 presenta las hipótesis utilizadas para estimar los costos mensuales de operación, considerando infraestructura, electricidad y mantenimiento.

La potencia de 1,2 kW se considera una hipótesis máxima de consumo continuo, no una medición promedio. Se calcula la energía con E = P × t, para 720 horas al mes. Con una tarifa ilustrativa de S/0,80/kWh, operar a 0,6 kW costaría S/345,60, a 0,9 kW S/518,40 y a 1,2 kW S/691,20 mensuales. El costo incremental atribuible al proyecto sería inferior si el equipo permaneciera encendido por otras tareas. Debe obtenerse una medición de consumo y una tarifa real antes de presentar estos valores como presupuesto definitivo.

Tabla 18

Escenarios de consumo eléctrico durante 30 días

Potencia media	Energía (kWh)	Gasto (S/)
0,6 kW	432	345,60
0,9 kW	648	518,40
1,2 kW	864	691,20

La Tabla 18 compara tres escenarios de consumo eléctrico durante un periodo de treinta días, considerando diferentes niveles hipotéticos de potencia.

Como referencia para valorar el tiempo de desarrollo se supone un profesional junior con remuneración mensual de S/2.500 y 160 horas de trabajo, equivalente a S/15,625 por hora. En una dedicación inicial de 60 horas, el costo económico del esfuerzo sería S/937,50; ocho horas de mantenimiento corresponderían a S/125 mensuales. Esta remuneración es una hipótesis presupuestal y no pretende afirmar que el equipo pagará un sueldo ni que esa cifra sea el promedio salarial oficial del mercado peruano. Antes de una entrega comercial sería indispensable cotizar personal y considerar beneficios laborales.

La factibilidad económica se limita a la validación académica y no incluye VAN, TIR o retorno de inversión, porque no se dispone de una organización cliente ni de ahorros operativos medidos. El modelo actual sería más sostenible si se activara MATTEOXPC únicamente durante sesiones de prueba, reduciendo horas de consumo eléctrico. Los costos de nube estarán condicionados por la tarifa efectiva de Azure y los créditos disponibles (Microsoft, s. f.).

12.3. Factibilidad operativa

La factibilidad operativa examina si el sistema puede utilizarse y mantenerse por personas concretas, no solo si existen CPU y memoria suficientes. El proceso previsto es breve: el administrador inicia sesión, incorpora documentos y verifica sus estados; el usuario formula preguntas sin crear una cuenta y revisa la fuente documental de la respuesta. Los responsables técnicos podrán diagnosticar errores mediante logs, revisar la red privada y restaurar archivos si se pierde información.

El principal límite operativo es que ProjectSevera se ejecutará en una computadora personal. Una interrupción eléctrica, cierre del programa o desconexión doméstica impedirá completar consultas RAG. Por tanto, el prototipo tendrá horarios de prueba declarados, un procedimiento de arranque y mensajes de indisponibilidad. Este uso no sustituye un servicio de soporte profesional atendido permanentemente; constituye una validación de una solución documental acotada.

El despliegue se considera operativamente viable en laboratorio si los responsables pueden realizar las operaciones de la sección 9 y recuperar el servicio después de una caída controlada. Los estudios sobre asistencia a operadores y sobre tutoría documental muestran aplicaciones posibles de RAG en contextos diferentes; sus resultados no pueden extrapolarse automáticamente a este grupo académico sin una evaluación propia (Barrionuevo-Valenzuela et al., 2026; Modran, 2025).

13. Conclusiones

El diseño delimita una aplicación de asistencia documental que distingue claramente el acceso a conocimiento escrito y la consulta de herramientas operativas. Los cuatro casos de uso se apoyan en ocho requerimientos funcionales concretos y once atributos de calidad medibles. Esta delimitación permite orientar la Unidad III hacia una implementación verificable, sin añadir funciones que no se puedan demostrar.

La arquitectura hexagonal separa la lógica de aplicación de los detalles de recuperación, persistencia y comunicación. Azure alojará la web, la administración documental y MCP, mientras

MATTEOXPC ejecutará ProjectSevera. La comunicación por Tailscale y el uso de archivos y metadatos evitan comprometer bases de datos y servidores adicionales que no son necesarios para el primer prototipo.

El diseño de seguridad prioriza amenazas relacionadas con documentos no confiables, permisos administrativos, herramientas MCP y disponibilidad de la computadora local. Los controles propuestos son proporcionales al entorno de laboratorio y deberán ser verificados mediante pruebas de acceso, inyección indirecta, restauración y caída de servicios. La existencia de diagramas o políticas escritas no constituye evidencia de implementación.

La factibilidad técnica y operativa es favorable para un prototipo con uso limitado, pero permanece condicionada al rendimiento real de Severa, la sincronización documental y la conectividad entre ambos nodos. El presupuesto identifica costos de nube, energía y trabajo como supuestos revisables, sin inventar ahorros ni un retorno financiero empresarial. Las pruebas de la Unidad III determinarán qué resultados cumplen los objetivos definidos.

14. Referencias bibliográficas

Adlakha, V., BehnamGhader, P., Lu, X. H., Meade, N., & Reddy, S. (2024). Evaluating correctness and faithfulness of instruction-following models for question answering. *Transactions of the Association for Computational Linguistics*, *12*, 681–699. https://doi.org/10.1162/tacl_a_00667

Anthropic. (2024, 25 de noviembre). *Introducing the Model Context Protocol*. https://www.anthropic.com/news/model-context-protocol

Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *The Twelfth International Conference on Learning Representations (ICLR 2024)*. https://doi.org/10.48550/arXiv.2310.11511

Barnett, S., Kurniawan, S., Thudumu, S., Brannelly, Z., & Abdelrazek, M. (2024). Seven failure points when engineering a retrieval augmented generation system. *Proceedings of the IEEE/ACM 3rd*

International Conference on AI Engineering – Software Engineering for AI, 194–199. https://doi.org/10.1145/3644815.3644945

Barrionuevo-Valenzuela, J., Calderón-González, D., Callejas, Z., & Griol, D. (2026). Supporting human operators during customer service interactions with agentic-RAG. *Proceedings of the 16th International Workshop on Spoken Dialogue System Technology*, 348–356. https://aclanthology.org/2026.iwsds-1.35/

Chen, T., Wang, H., Chen, S., Yu, W., Ma, K., Zhao, X., Zhang, H., & Yu, D. (2024). Dense X retrieval: What retrieval granularity should we use? *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 15159–15177. https://doi.org/10.18653/v1/2024.emnlp-main.845

Cockburn, A. (2005). *Hexagonal architecture (Ports and Adapters)*. Alistair.cockburn.us. https://alistair.cockburn.us/hexagonal-architecture/

Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., & Tramèr, F. (2024). AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *Advances in Neural Information Processing Systems, 37*. https://arxiv.org/abs/2406.13352

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Osazuwa Ness, R., & Larson, J. (2024). From local to global: A graph RAG approach to query-focused summarization. arXiv. https://doi.org/10.48550/arXiv.2404.16130

Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2024). RAGAs: Automated evaluation of retrieval augmented generation. *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL 2024)*, 150–158. Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.eacl-demo.16

Feng, S., Patel, S., Wan, H., & Joshi, S. (2021). MultiDoc2Dial: Modeling dialogues grounded in multiple documents. *Proceedings of the 2021 Conference on Empirical Methods in Natural Language*

Processing (EMNLP 2021), 6162–6176. Association for Computational Linguistics. https://doi.org/10.18653/v1/2021.emnlp-main.498

Feng, S., Wan, H., Gunasekara, C., Patel, S., Joshi, S., & Lastras, L. (2020). doc2dial: A goal-oriented document-grounded dialogue dataset. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP 2020)*, 8118–8128. Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.emnlp-main.652

Friel, R., Belyi, M., & Sanyal, A. (2024). RAGBench: Explainable benchmark for retrieval-augmented generation systems. *arXiv preprint arXiv:2407.11005*. https://doi.org/10.48550/arXiv.2407.11005

Gao, L., Ma, X., Lin, J., & Callan, J. (2023). Precise zero-shot dense retrieval without relevance labels. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 1762–1777. https://doi.org/10.18653/v1/2023.acl-long.99

Gao, T., Yen, H., Yu, J., & Chen, D. (2023). Enabling large language models to generate text with citations. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 6465–6488. https://doi.org/10.18653/v1/2023.emnlp-main.398

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2024). Retrieval-augmented generation for large language models: A survey. *arXiv*. https://doi.org/10.48550/arXiv.2312.10997

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. En *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec 2023)* (pp. 79–90). ACM. https://doi.org/10.1145/3605764.3623985

Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M.-W. (2020). Retrieval augmented language model pre-training. En *Proceedings of the 37th International Conference on Machine Learning (PMLR)* (Vol. 119, pp. 3929–3938). https://proceedings.mlr.press/v119/guu20a.html

Hasan, M. M., Li, H., Fallahzadeh, E., Rajbahadur, G. K., Adams, B., & Hassan, A. E. (2025). Model Context Protocol (MCP) at first glance: Studying the security and maintainability of MCP servers. *arXiv*. https://doi.org/10.48550/arXiv.2506.13538

Hou, X., Zhao, Y., Wang, S., & Wang, H. (2026). Model Context Protocol (MCP): Landscape, security threats, and future research directions. *ACM Transactions on Software Engineering and Methodology, 35*(10), Artículo 302. https://doi.org/10.1145/3796519

International Organization for Standardization & International Electrotechnical Commission. (2022). *Information security, cybersecurity and privacy protection — Guidance on managing information security risks* (ISO/IEC 27005:2022). ISO. https://www.iso.org/standard/80585.html

International Organization for Standardization & International Electrotechnical Commission. (2023). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model* (ISO/IEC 25010:2023). ISO. https://www.iso.org/standard/78176.html

Izacard, G., & Grave, E. (2021). Leveraging passage retrieval with generative models for open domain question answering. *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume (EACL 2021)*, 874–880. Association for Computational Linguistics. https://doi.org/10.18653/v1/2021.eacl-main.74

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, *55*(12), Artículo 248, 1–38. https://doi.org/10.1145/3571730

Jiang, Z., Xu, F. F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Callan, J., & Neubig, G. (2023). Active retrieval augmented generation. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, 7969–7992. Association for Computational Linguistics. https://doi.org/10.18653/v1/2023.emnlp-main.495

Johnson, J., Douze, M., & Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data, 7*(3), 535–547. https://doi.org/10.1109/TBDATA.2019.2921572

Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W.-t. (2020). Dense passage retrieval for open-domain question answering. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP 2020)*, 6769–6781. Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.emnlp-main.550

Khattab, O., & Zaharia, M. (2020). ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 39–48. https://doi.org/10.1145/3397271.3401075

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, *33*, 9459–9474. https://doi.org/10.48550/arXiv.2005.11401

Li, M., Zhao, Y., Yu, B., Song, F., Li, H., Yu, H., Li, Z., Huang, F., & Li, Y. (2023). API-Bank: A comprehensive benchmark for tool-augmented LLMs. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, 3102–3116. Association for Computational Linguistics. https://doi.org/10.18653/v1/2023.emnlp-main.187

Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *42*(4), 824–836. https://doi.org/10.1109/TPAMI.2018.2889473

Microsoft. (s. f.). *What is Azure Virtual Network?* Microsoft Learn. https://learn.microsoft.com/en-us/azure/virtual-network/virtual-networks-overview

Model Context Protocol. (2025). *Model Context Protocol specification (2025-06-18).* https://modelcontextprotocol.io/specification/2025-06-18

Modran, H. A. (2025). Leveraging RAG with ACP & MCP for adaptive intelligent tutoring. *Applied Sciences, 15*(21), Artículo 11443. https://doi.org/10.3390/app152111443

Muennighoff, N., Tazi, N., Magne, L., & Reimers, N. (2023). MTEB: Massive Text Embedding Benchmark. *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL 2023)*, 2014–2037. Association for Computational Linguistics. https://doi.org/10.18653/v1/2023.eacl-main.148

National Institute of Standards and Technology. (2020). *Security and privacy controls for information systems and organizations* (NIST Special Publication 800-53, Rev. 5). U.S. Department of Commerce. https://doi.org/10.6028/NIST.SP.800-53r5

Nogueira, R., Jiang, Z., Pradeep, R., & Lin, J. (2020). Document ranking with a pretrained sequence-to-sequence model. *Findings of the Association for Computational Linguistics: EMNLP 2020*, 708–718. Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.findings-emnlp.63

Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Hong, L., Tian, R., Xie, R., Zhou, J., Gerstein, M., Li, D., Liu, Z., & Sun, M. (2024). ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *The Twelfth International Conference on Learning Representations (ICLR 2024)*. https://openreview.net/forum?id=dHng2O0Jjr

Ribeiro, M. T., Wu, T., Guestrin, C., & Singh, S. (2020). Beyond accuracy: Behavioral testing of NLP models with CheckList. *Proceedings of the 58th Annual Meeting of the Association for*

Computational Linguistics (ACL 2020), 4902–4912. Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.acl-main.442

Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, *3*(4), 333–389. https://doi.org/10.1561/1500000019

Saad-Falcon, J., Khattab, O., Potts, C., & Zaharia, M. (2024). ARES: An automated evaluation framework for retrieval-augmented generation systems. *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics*, 338–354. https://doi.org/10.18653/v1/2024.naacl-long.20

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, *36*, 68539–68551. https://doi.org/10.48550/arXiv.2302.04761

Shuster, K., Poff, S., Chen, M., Kiela, D., & Weston, J. (2021). Retrieval augmentation reduces hallucination in conversation. *Findings of the Association for Computational Linguistics: EMNLP 2021*, 3784–3803. Association for Computational Linguistics. https://doi.org/10.18653/v1/2021.findings-emnlp.320

Spring Security. (s. f.). *In-memory authentication.* Spring. https://docs.spring.io/spring-security/reference/servlet/authentication/passwords/in-memory.html

Tailscale. (s. f.). *Access controls.* https://tailscale.com/kb/1018/acls

Yan, S.-Q., Gu, J.-C., Zhu, Y., & Ling, Z.-H. (2024). Corrective retrieval augmented generation. *arXiv preprint arXiv:2401.15884*. https://doi.org/10.48550/arXiv.2401.15884

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *The Eleventh International Conference on Learning Representations (ICLR 2023)*. https://doi.org/10.48550/arXiv.2210.03629

Zhan, Q., Liang, Z., Ying, Z., & Kang, D. (2024). InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. *Findings of the Association for Computational Linguistics: ACL 2024*, 10471–10506. Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.findings-acl.624